# National University of Computer & Emerging Sciences

## Lecture 20

## Synchronization Issues

# The Synchronization Problem

- Processes may be **independent** or **cooperating**
- Cooperating process can affect or be affected by other processes

- Cooperating process can share data
  - Inter-Process communication in case of heavy-weight processes
  - Same logical address space in case of threads

# The Synchronization Problem

- Multiple processes may be sharing a common storage
  - Common Main memory
  - Or Common File
  - Or Common xyz
- The nature of the common storage does not change the nature of the problem
- “Common storage” is an abstract concept
- Implementation may differ

# The Synchronization Problem

- Concurrent access to shared data may result in data inconsistency

- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes

# The Synchronization Problem

- One process/thread should not get into the way of another process/thread when doing critical activities

- Proper sequence of execution should be followed when dependencies are present
  - A produces data, B prints it
  - Before printing B should wait while A is producing data
  - B is dependent on A

# The Synchronization Problem

- The problem relates to both Threads and Processes as data sharing can be using
  - IPC in case of heavy-weight processes
  - Same logical address space in case of threads

- Same solutions exists
- The only difference could be the level at which the solution is applied
  - Kernel level
  - User level
- From now on threads and processes both mean the same i.e. “Execution path”, unless otherwise specified

# If there is no synchronization



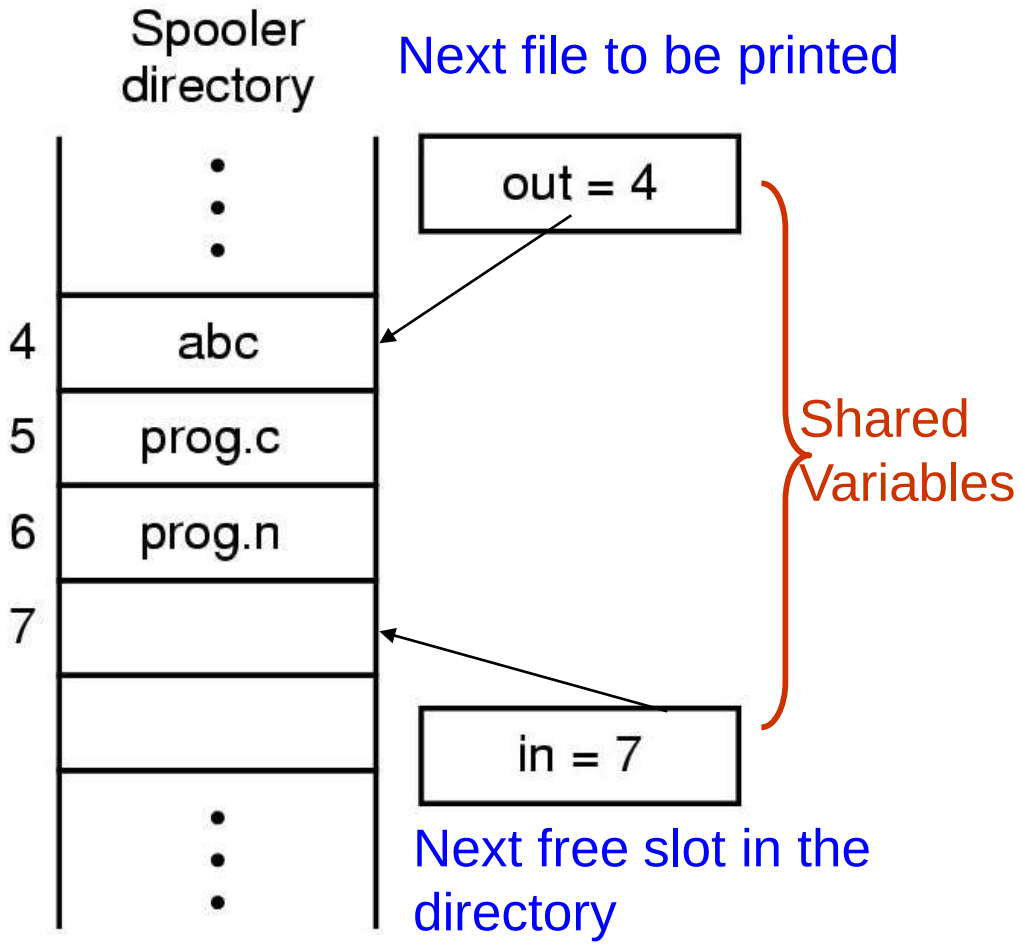
# Example Print Spooler

- If a process wishes to print a file it adds its name in a **Spooler Directory**
- The **Printer process**
  - Periodically checks the spooler directory
  - Prints a file
  - Removes its name from the directory

# Example Print Spooler

If any process wants to print a file it will execute the following code

1. Read the value of `in` in a local variable `next_free_slot`
2. Store the name of its file in the `next_free_slot`
3. Increment `next_free_slot`
4. Store back in `in`



Let A and B be processes wh… their files

Process A

1. Read the value of in in a local variable next_free_slot

2. Store the name of its fi… in the next_free_slot

3. Increment next_free_slot

4. Store back in in

Process B

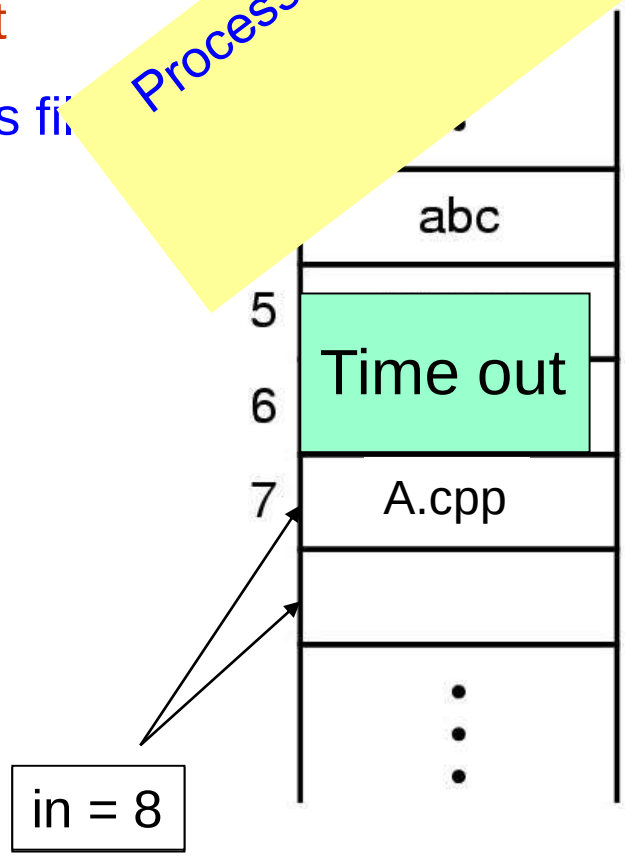
1. Read the value of in in a loc… variable next_free_slot

2. Store the name of its file in the next_free_slot

3 .Increment next_free_slot

4. Store back in in

Process B will never receive any output




$next_free_slot_a = 7$

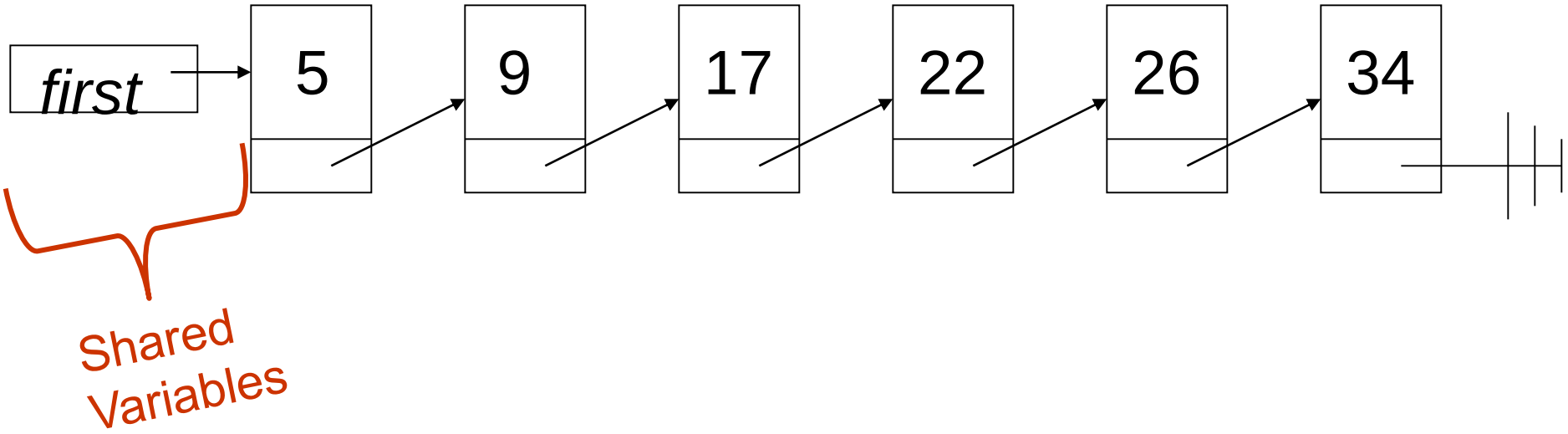
$next_free_slot_b = 7$

# Race condition

- A situation where several processes access and manipulate the same data concurrently, and the outcome of the execution depends on the particular order in which the access takes place, is called **race condition.**
  - Debugging is not easy
  - Most test runs will run fine

# Other examples of Race conditions

- Adding node to a shared linked list



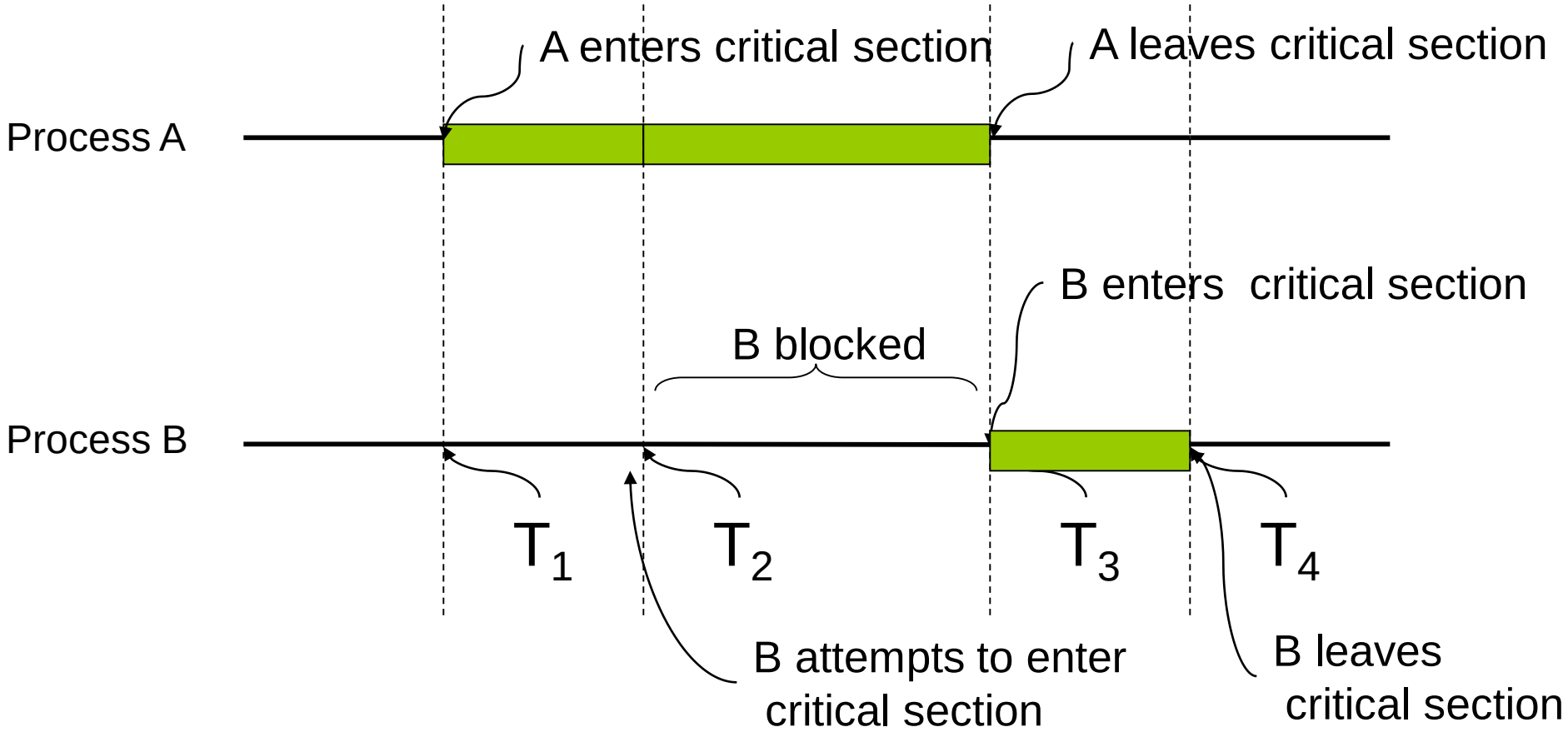
# Reason behind Race Condition

- Process B started using one of the shared variables before process A was finished with it.
- At any given time a Process is either
  - Doing internal computation => no race conditions
  - Or accessing shared data that can lead to race conditions
- Part of the program where the shared memory is accessed is called **Critical Region**
- Races can be avoided
  - **If no two processes are in the critical region at the same time.**

# Critical Section

```
void threadRoutine()
{
   int y, x, z;
   y = 3 + 5x;
   y = Global_Var;
   y = y + 1;
   Global_Var = y;
   …
   …
}
```

# Critical Section




Mutual Exclusion

At any given time, only one process is in the critical section

# Critical Section

- Avoid race conditions by not allowing two processes to be in their critical sections at the same time
- We need a mechanism of mutual exclusion
- Some way of ensuring that one processes, whilst using the shared variable, does not allow another process to access that variable